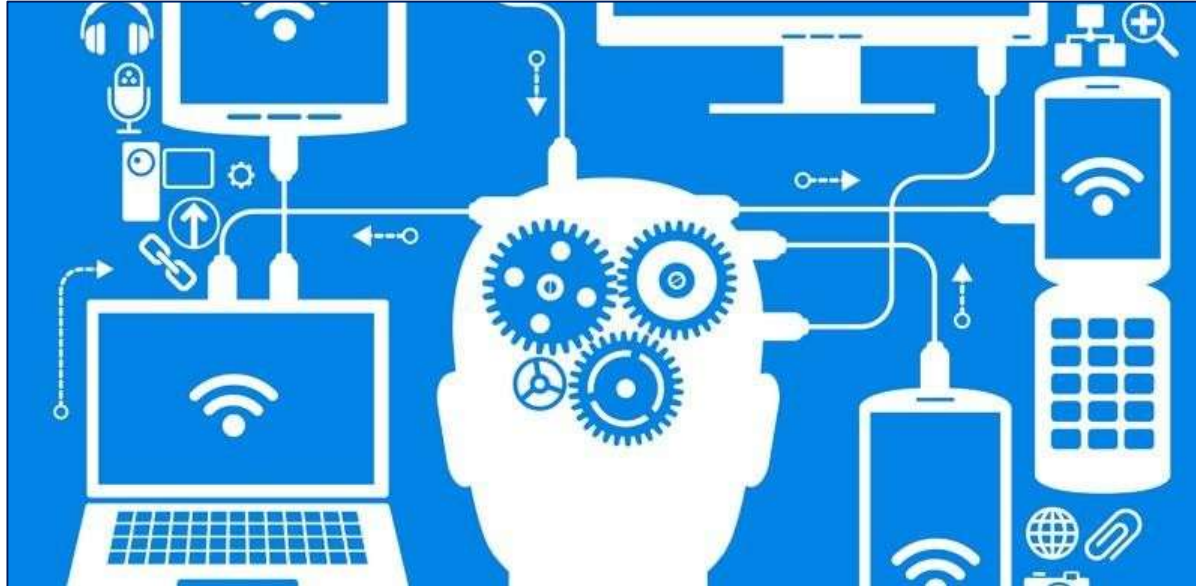


Software Engineering - 5CM505

Lecture 6b – Object-Oriented Analysis & Design

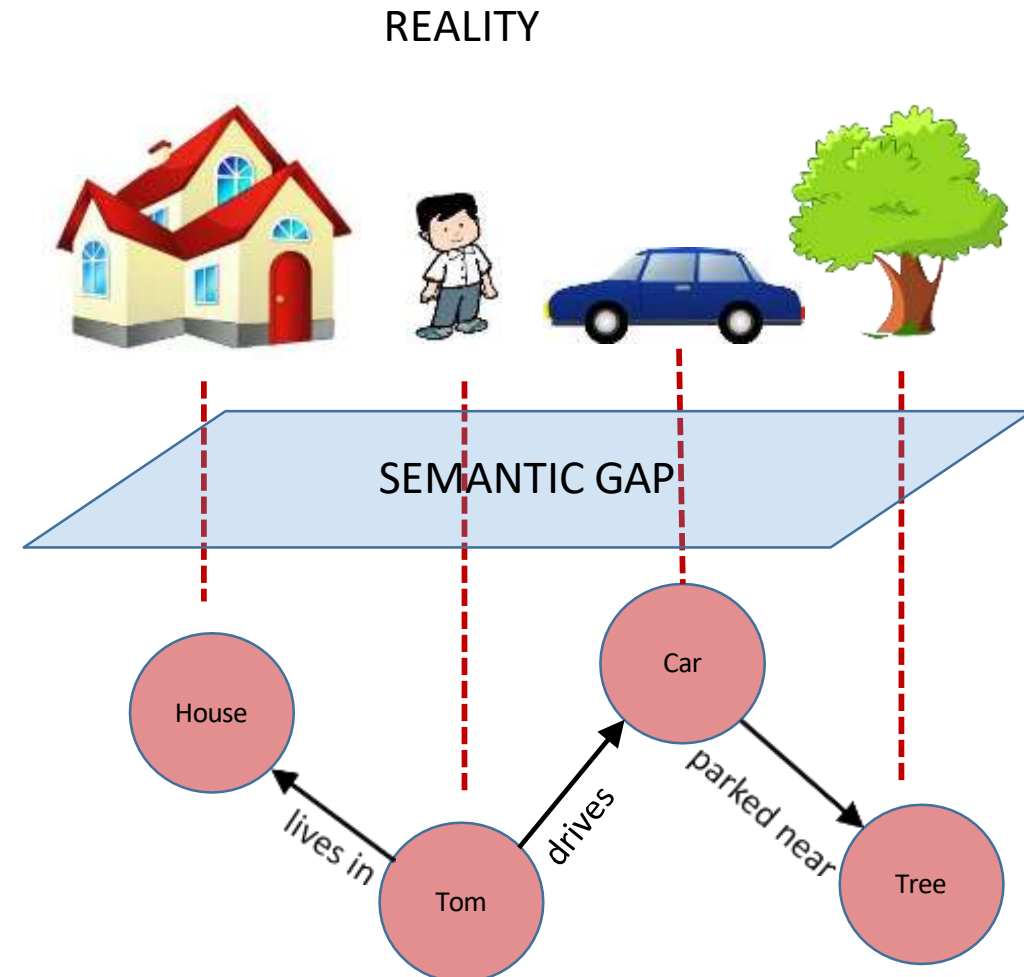
Ioannis Tsioulis (MC)



slides acknowledgements:
Dr Amanda Whitbrook (UoD)
Dr Saeed Ullah (UoD)

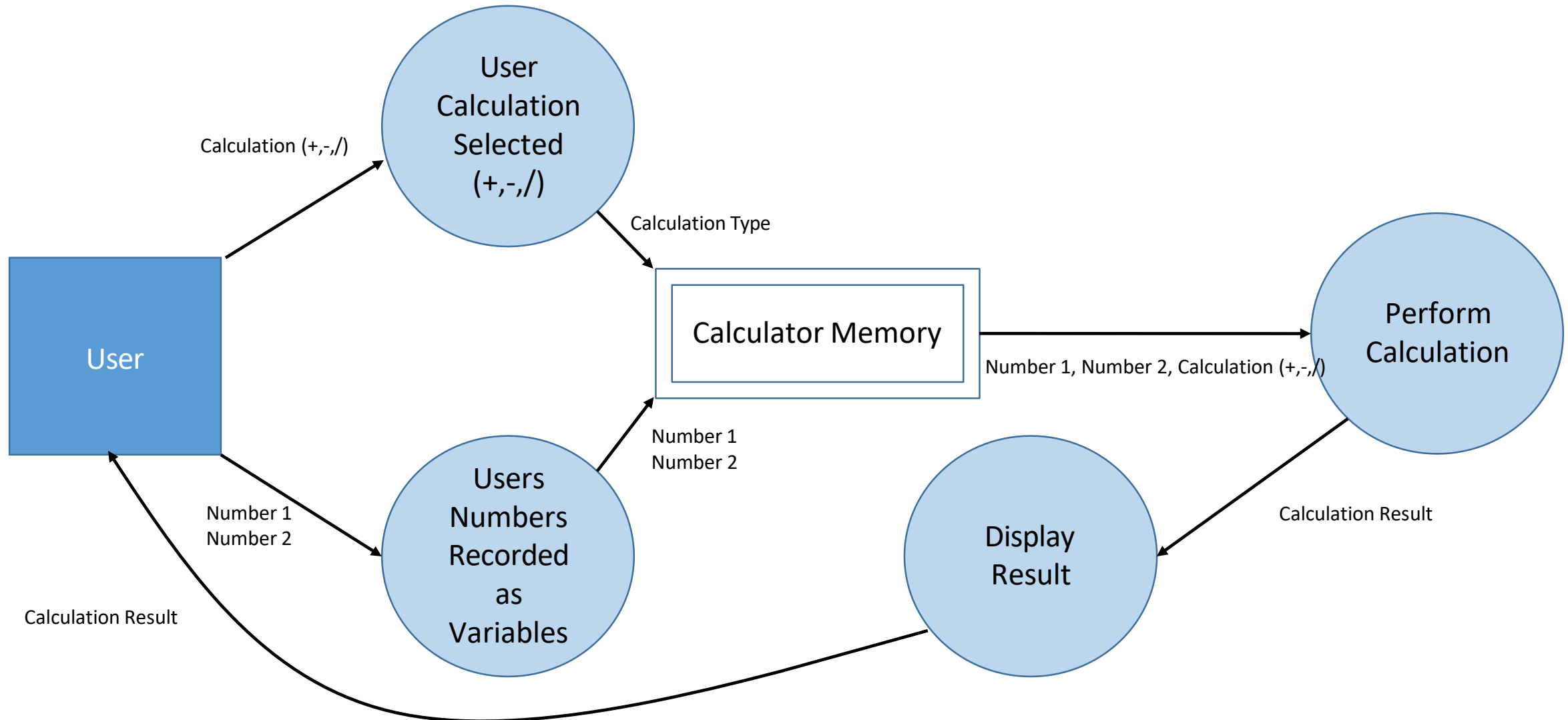
Lecture 6 - Content

- Recap from last week
- System Development Methods
 - Functional / data oriented
 - Object-oriented
- Object-oriented Analysis
- Object-oriented Construction
- OOP and Coupling and Cohesion
- Summary



Recap from Last Week

Function Flow Diagram



Functional Modelling

ADVANTAGES

- **Shows relationships between functions**
- Simple technique, which means that the whole team can be involved in the process
- Provides a common understanding of the system
- Customers can also get involved
- Oversights and errors can be detected early
- Identifies the external parts of the system

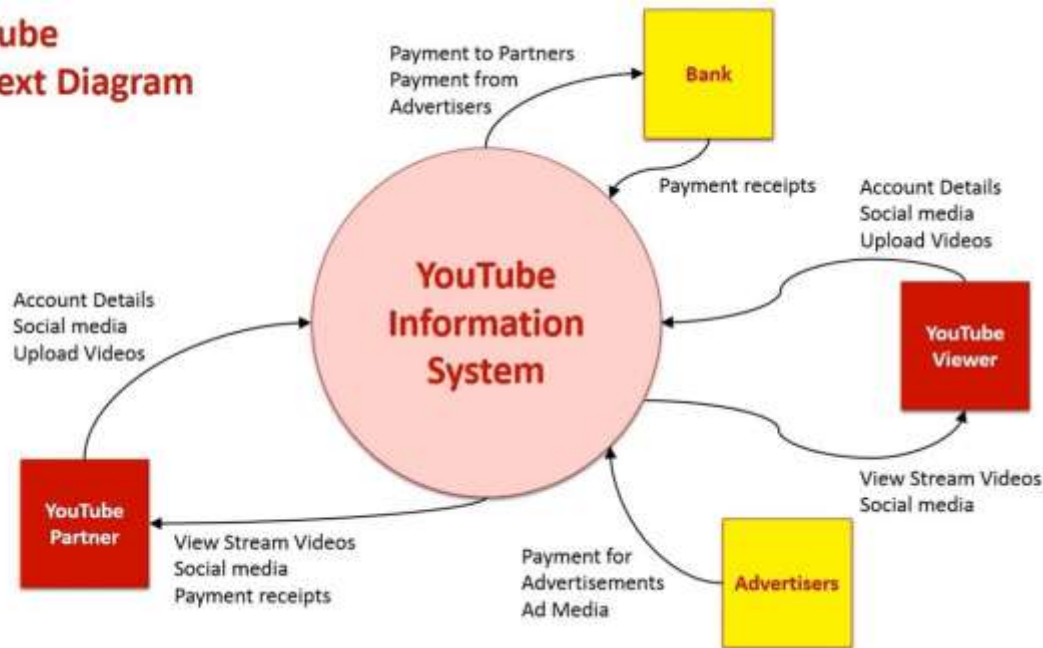
DISADVANTAGES

- Difficult to verify completeness
- Diagrams can get complicated if not broken down into separate levels properly
- **Functions and data are treated as separate**

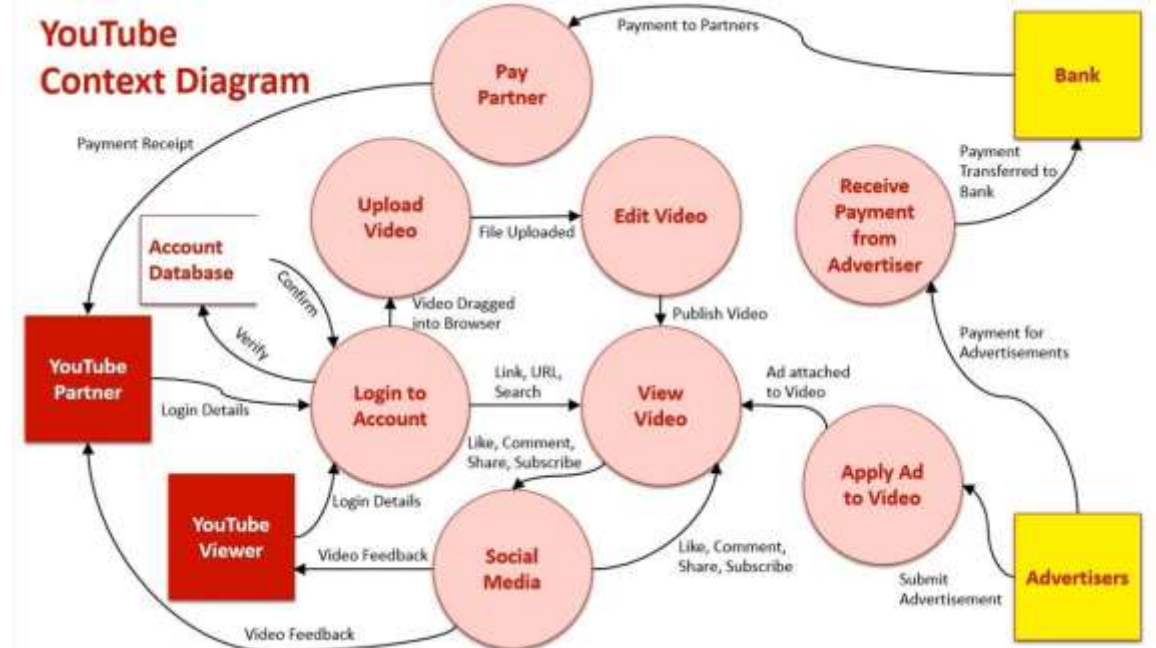


FFDs & Context Diagrams

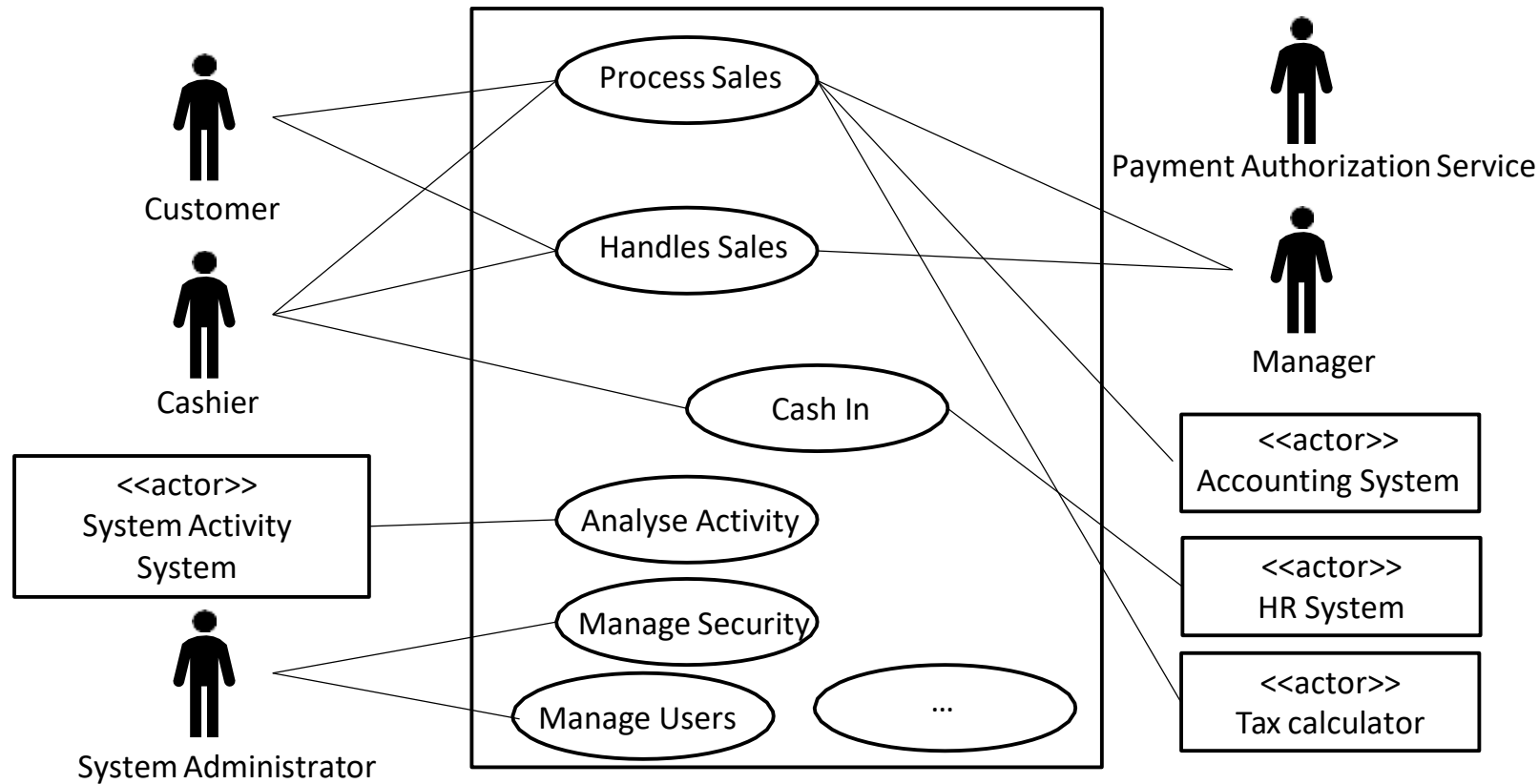
**YouTube
Context Diagram**



**YouTube
Context Diagram**



Use Cases & Context Diagrams

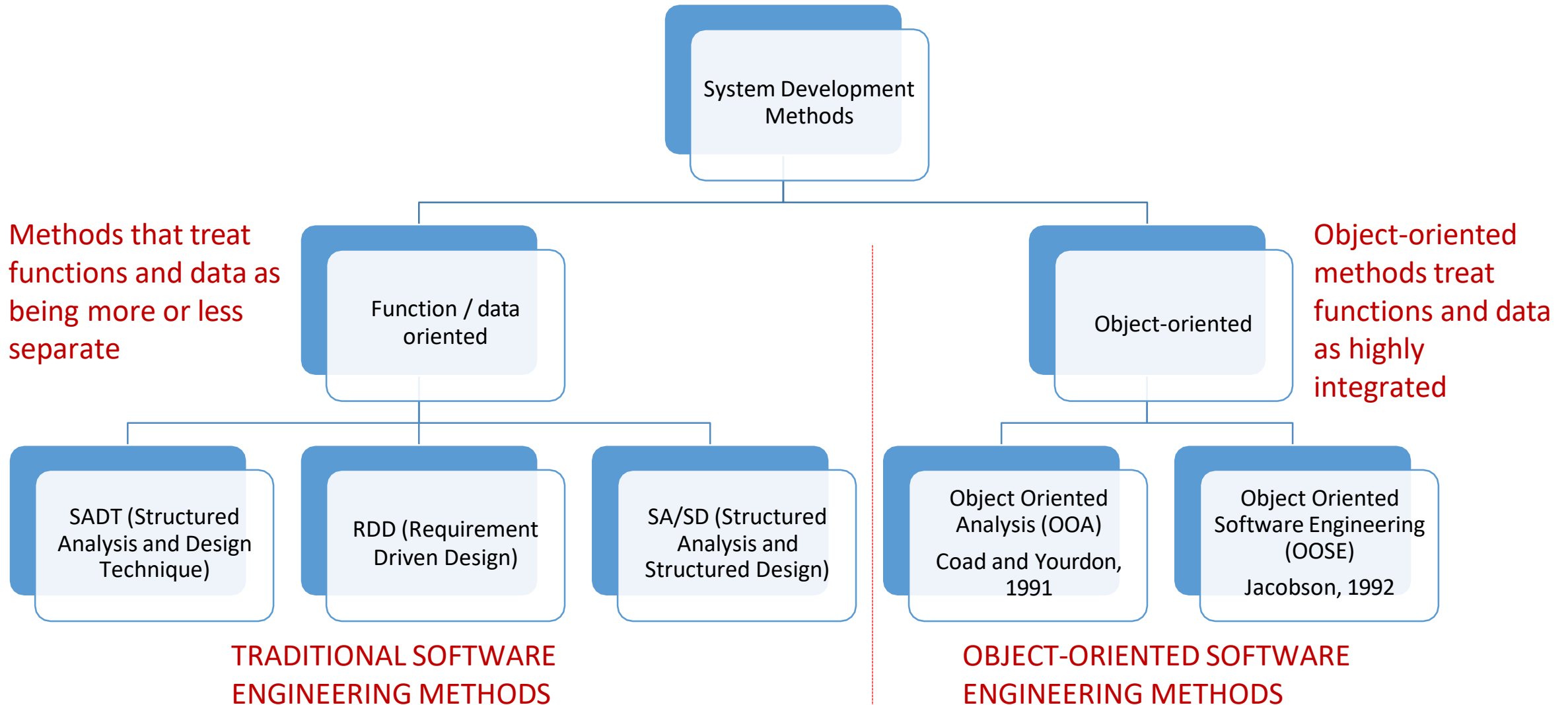


Learning Outcomes

- By the end of this lecture you should be able to:
 - Explain the **principles behind object-oriented modelling**, and use it to aid with designing software systems
 - Describe the differences between **functional (data) modelling and object-oriented modelling**
 - Discuss the **advantages of object-oriented modelling and analysis**
 - Explain the **OO principles** of *encapsulation*, *information hiding*, *inheritance* and *polymorphism*

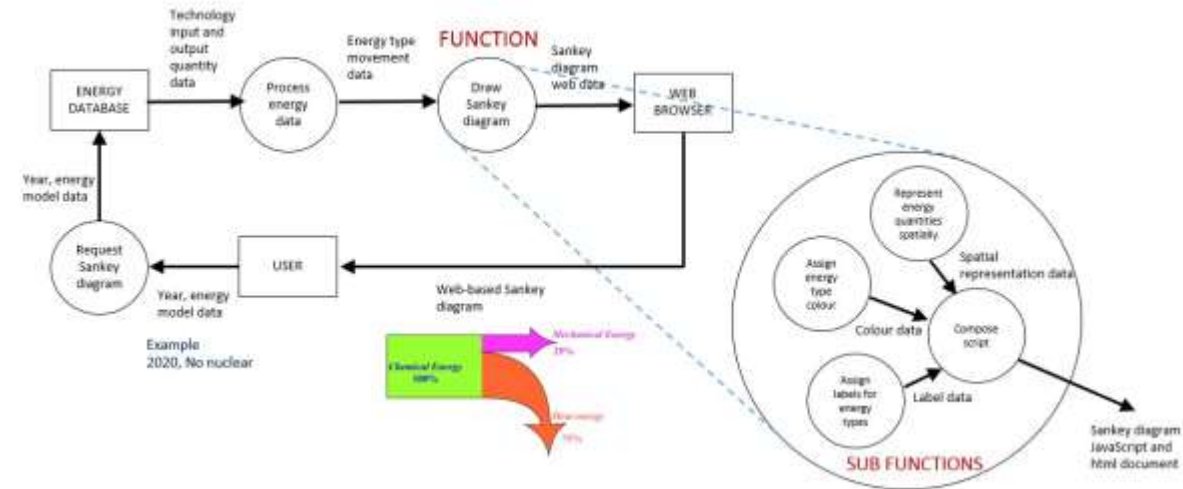
System Development Methods

System Development Methods



Function/Data Development Methods

- These methods distinguish between **functions** and **data**
- **Functions** seen as **active**, with a given behaviour; **data** seen as a **passive** holder of information that is affected by functions
- System thus typically broken down into functions, with data sent between those functions
- Design is based around how a certain behaviour will be carried out with functions eventually converted into source code



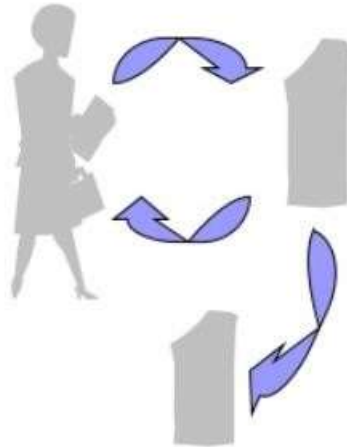
- However, the paradigm is an artefact of computing
- Assembly language divides instructions and data, and memory has separate spaces for instructions and data in modern processors
- High level languages tended to follow suit
- Object-orientation is an attempt to escape from this approach

Drawbacks of Separating Functions and Data

- A major problem is that, in general, *all functions must know the data structure*, which means that it has to be hard-coded into the function
- If we want to *modify the data structure*, we then have to *modify all the functions* relating to it
- Slight modifications to the data structure can have a major impact on the code, often making it unstable. This means systems developed using the functional / data method can be difficult to maintain
- The function / data method separates everything into either data or functions, which is not natural. It requires a translation when we start to think about how the system will work. This is known as a **semantic gap** between the model and reality
- Also, **requirements are often captured in natural language** – what exists in the system and what it will do

Functional Thinking vs OO Thinking

Functional



Withdraw, deposit, transfer

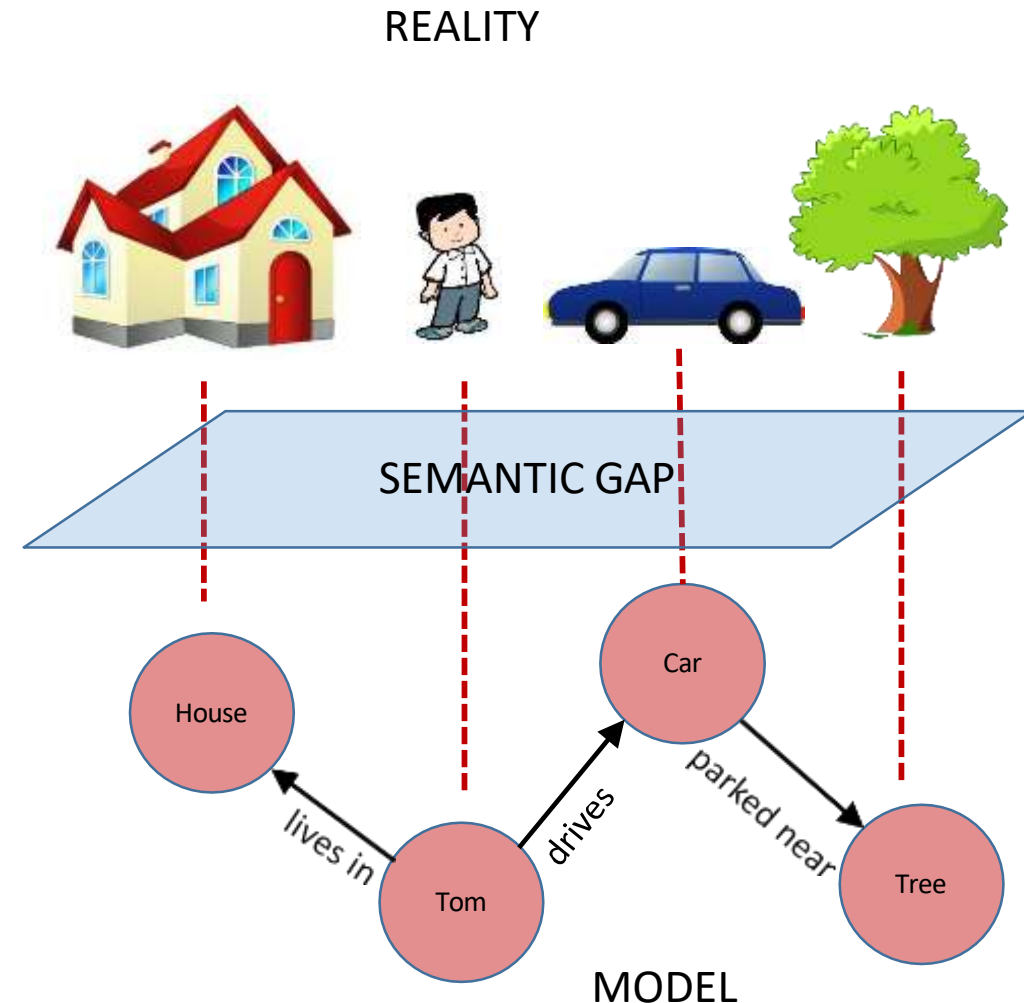
Object Oriented



Customer, money, account

Object Orientation

- A technique for system modelling where the system has a number of **objects** that **interact**. It is independent of any programming language
- A model designed using object-orientation is often easy to understand as it can be *directly related to reality*. Often there is only a small gap between reality and the model.



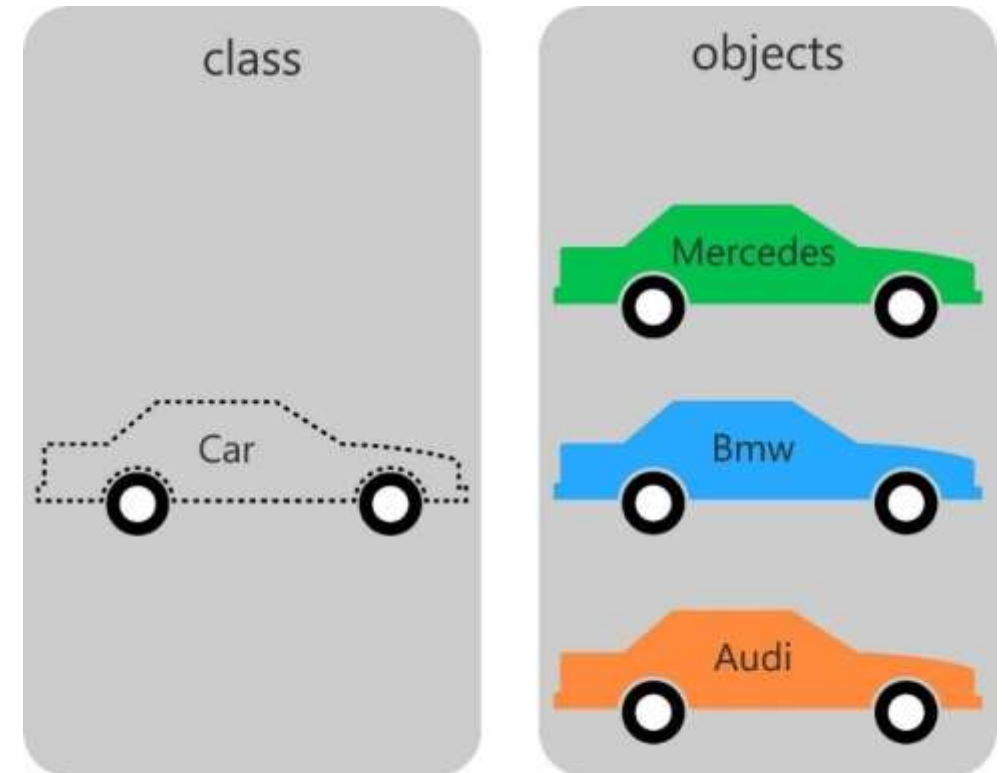
Addressing the Drawbacks of Functional Modelling

- With object-orientation the requirements are modelled from the natural language
- What **exists** in the system is modelled as an **object** and what it **does** is modelled as a **method** of that object
- The objects that exist in the system are usually stable and change very little
- When changes are needed, they usually only affect one or a few objects, which means that, usually, only small changes are necessary
- If we wish to have a stable system, we should consider what tends to change and then design according to this knowledge

Object-oriented Analysis

Object-oriented Analysis

- The purpose, as with all other analysis methods, is to understand the application in terms of the system's functional requirements
- The difference is the means of expression
- **Function / data analysis** considers the system's data and behaviour separately
- **Object-oriented analysis** combines them and regards them as integrated objects
- Object-oriented analysis consists of the following five steps:
 1. Identifying the objects
 2. Organising the objects
 3. Describing how they interact
 4. Defining the operations on the objects
 5. Defining the objects internally

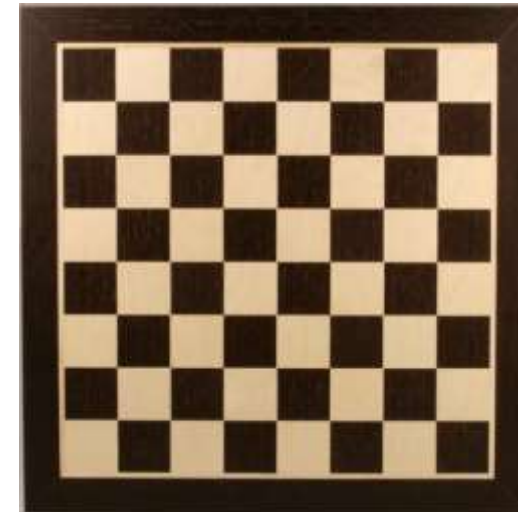


1. Identifying the Objects

- The first step in OO analysis is to identify all the objects in the problem domain
- These can be found by examining all the **nouns** within this domain



king queen bishop knight rook pawn



board



player1



player2

2. Organising the Objects

- Different object types can be organised according to different criteria
- Examples:
 - Active / passive
 - Physical / conceptual
 - Temporary / permanent
 - Part / whole
 - Generic / specific
 - Private / public
 - Shared / non-shared

- Identifying **which objects work together**
- Deciding whether an **object is part of another**, e.g. a house can be composed of doors, windows, bricks etc.
- Deciding which **objects are dependent** on others

2. Organising the Objects

- Another way of organising them is to consider how **similar** they are to each other
- If they are similar they may belong to the same **class**

Class

For example, the generic 'chess piece' is a physical game piece that is capable of moving on the chess board somehow and is either black or white in colour



Chess piece

Class

Sub-classes

Inherit the characteristics of the class that they belong to, so each of them moves on the chess board and is either black or white in colour

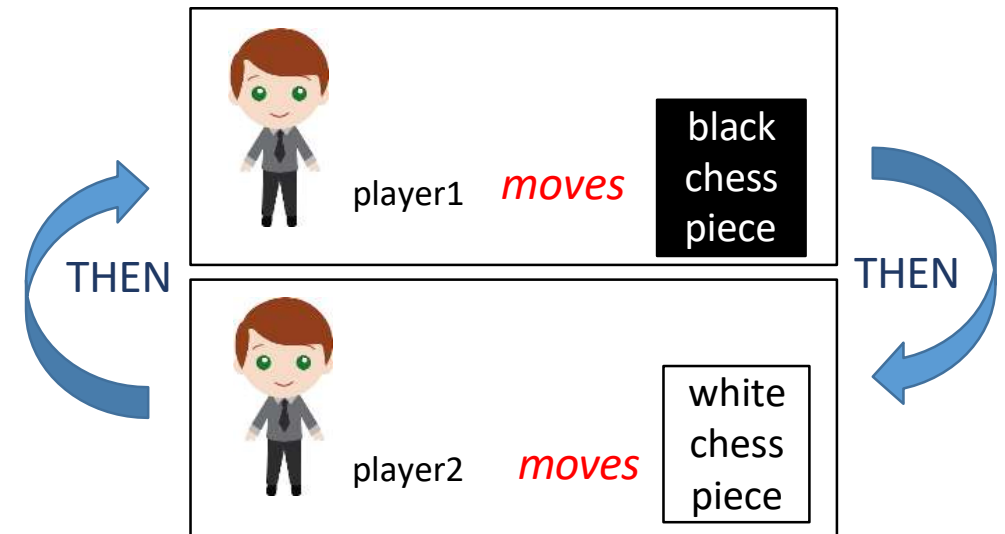


king queen bishop knight rook pawn

Sub-classes

3. Describing Object Interactions

- Next, we need to think about how the objects **interact** with each other.
- We can describe different scenarios or ***use cases*** in which the objects take part and communicate with each other
- This enables us to determine *what the other objects expect from a given object*



4. Operations on Objects

- We need to consider **what can be done** with each object in our system considering operations.
- The **operations** could be primitive, for example, *create, add, delete*
- Or they could be more complex, for example, assembling a report based on information from other objects
- The operations we identify are called **methods**



In our chess example we identified that the class 'chess piece' has a method 'move'

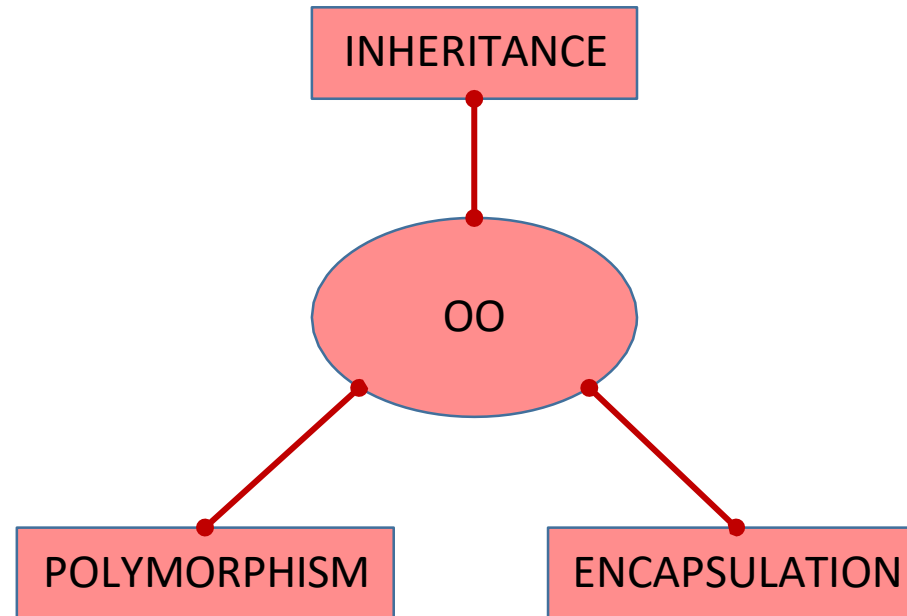
5. Defining Objects Internally

- This means defining the **information** that each object must hold
- To do this we **assign properties** to each class type

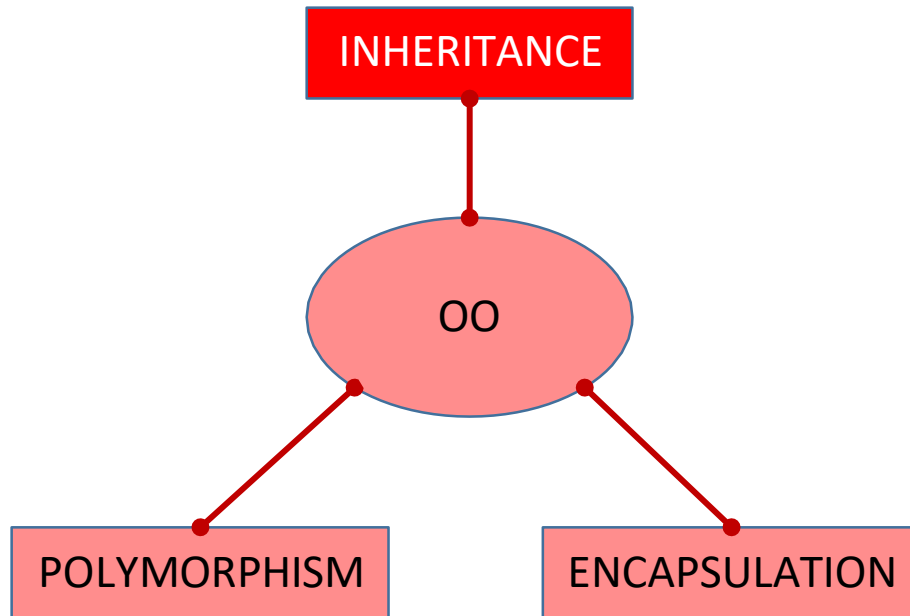


In our chess example we identified that the class 'chess piece' has property 'colour'

Principles of Object-oriented Design



Inheritance



- When a sub-class **inherits** from another class it inherits all of its **properties** and **methods**
- In our chess example the sub-class bishop inherits all of the properties and methods of the base class 'chess piece', i.e. the bishop has a colour property and move method



Chess piece

Class

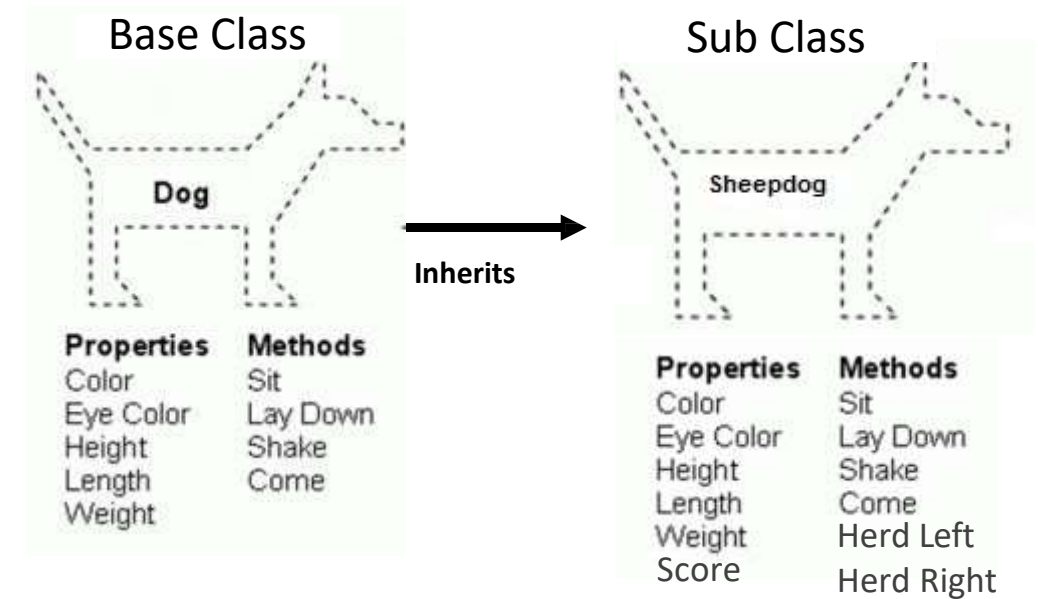


king queen bishop knight rook pawn

Sub-classes

Inheritance

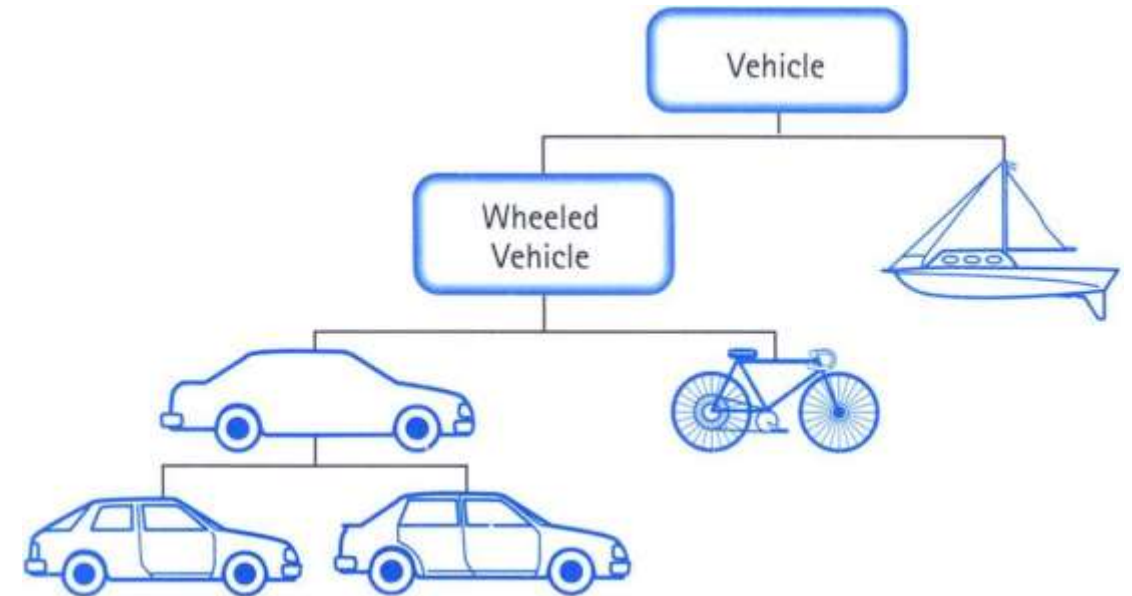
- With **Inheritance** you may create a more *specialised version* of the class
- The **sub class** can add new properties and methods
- For example, we could add a *Score* property to store the sheepdog's performance in sheep trials
- We could add methods for rounding up sheep.



- Inheritance has many advantages.
- What are they?

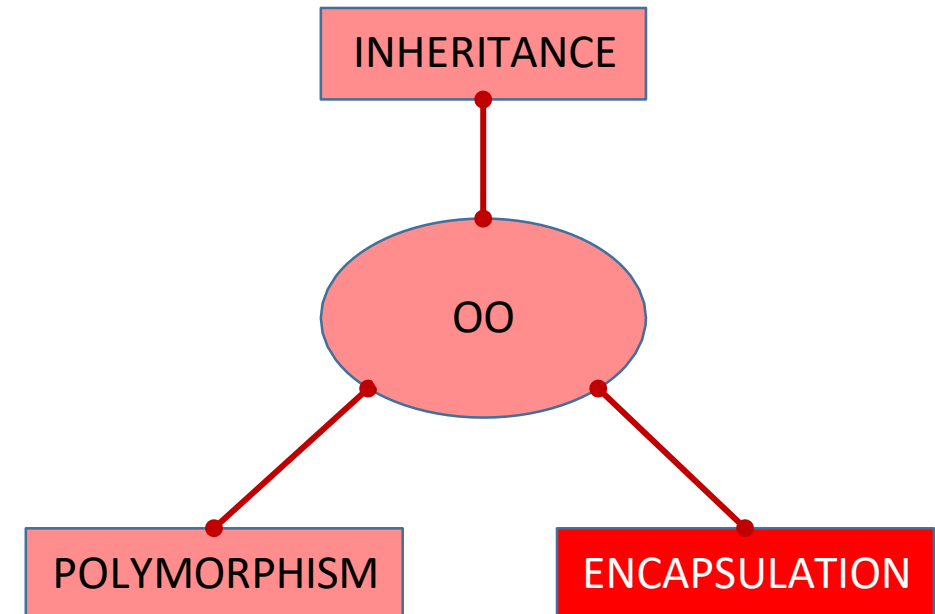
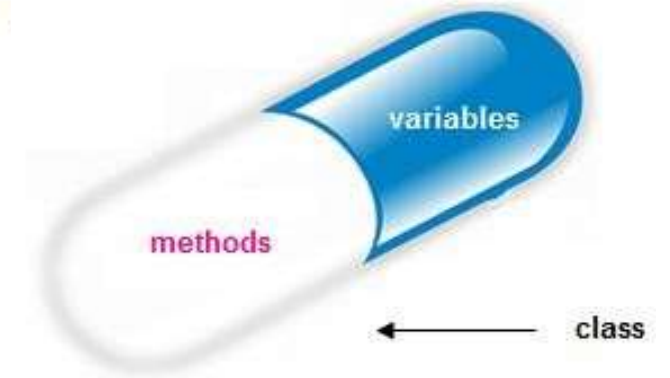
Inheritance - Advantages

- Forces a more thorough data analysis
- Reduces development time
- Reduces code length
- Avoids redundancy
- Reduces errors as code is reused more
- Consequently saves money



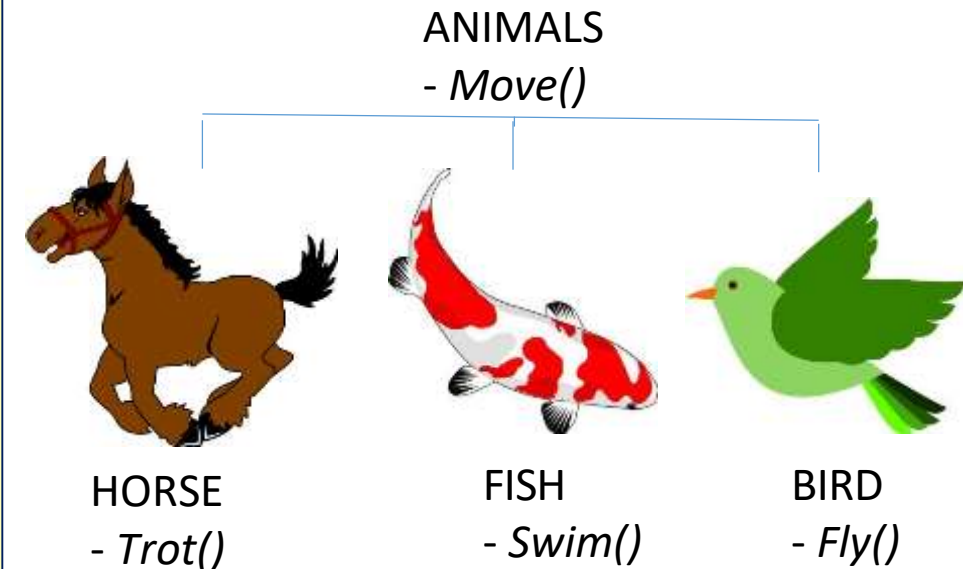
Encapsulation

- We have seen that object-orientation binds together the data (attributes / properties) and the functions (methods) that manipulate it, within a class. This is known as **encapsulation**.
- Encapsulation keeps both the attributes and methods safe from outside interference and misuse leading to the important OO concept of **information hiding**.
- The *internal implementation* of a class is hidden from the rest of the program.
- Encapsulation thus helps to create code that is **loosely coupled**.
- The ability of other objects to modify an object's state and behaviour is reduced because the details are hidden



Polymorphism

- Sub-classes can possess methods bearing the same name as those in the base class, but each may have different behaviours.
- As an example, let us assume there is a base class named Animals from which the subclasses Horse, Fish and Bird are derived.
- Let us also assume that the Animals class has a function named Move, which is inherited by all sub-classes mentioned.
- With **polymorphism**, each subclass may have its own way of implementing the function.
- So, for example, when the Move function is called in an object of the Horse class, the function might respond by displaying trotting on the screen.
- On the other hand, when the same function is called in an object of the Fish class, swimming might be displayed on the screen. In the case of a Bird object, it may be flying.



Polymorphism

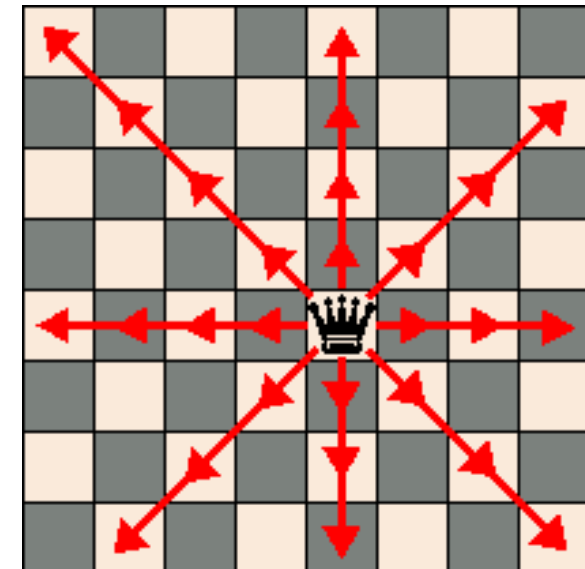
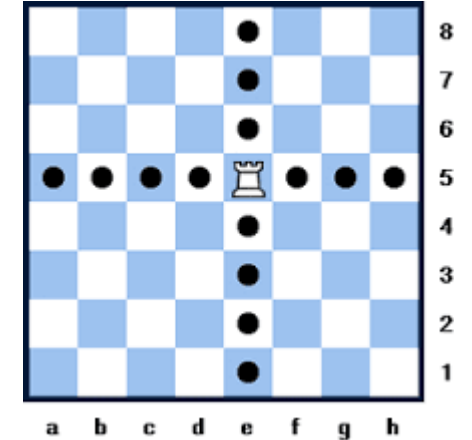
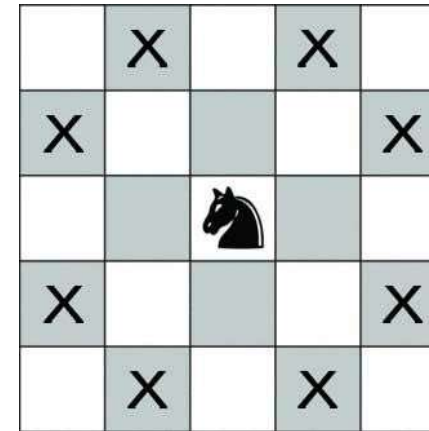


king queen bishop knight rook pawn

In our chess example we identified that the class 'chess piece' has sub-classes:

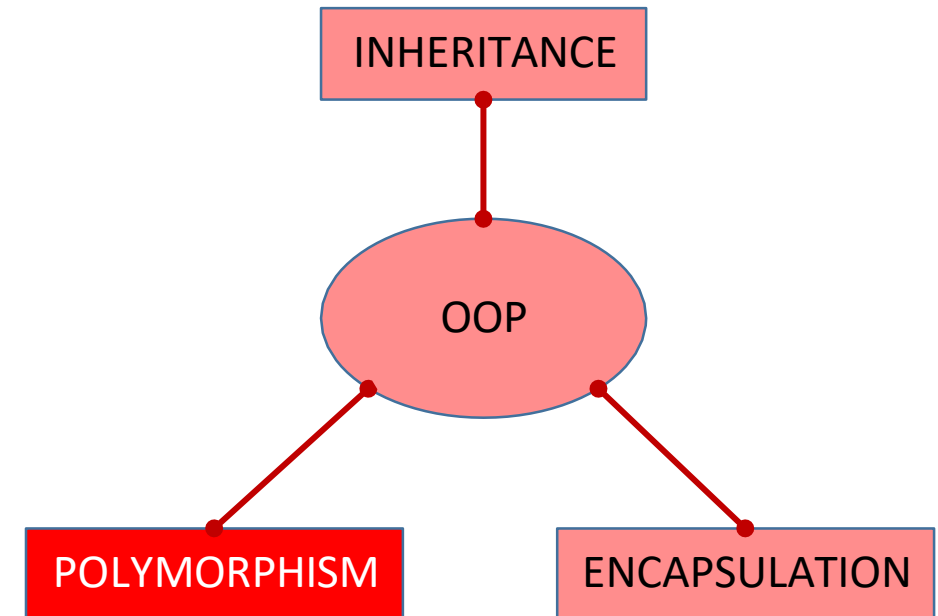
- King
- Queen
- Bishop
- Knight
- Rook
- Pawn

Each piece has the method 'move', but we are not restricted to making that method the same for each sub-class. For example, the queen moves in an entirely different way to the knight or rook



Polymorphism

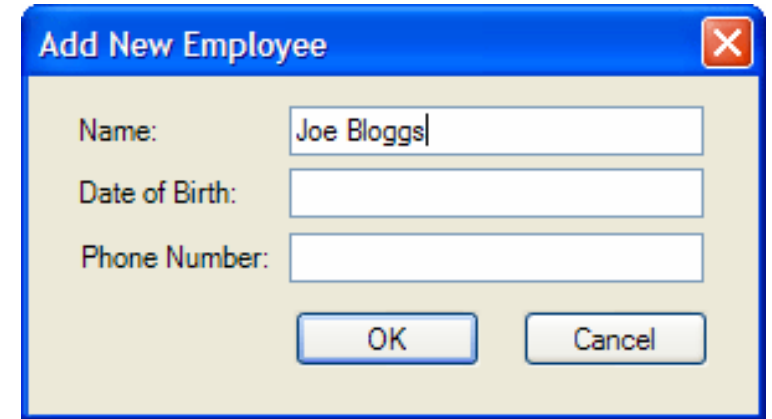
- In effect, **polymorphism** trims down the work of the developer.
- Can initially create a general class with all the attributes and behaviours necessary.
- When more specific subclasses are needed, with certain unique attributes and behaviours, the developer can simply alter code in the specific portions where the behaviours will differ.
- All other portions of the code can be left alone



Object-oriented Construction

Object Oriented Programming

- Object-oriented programming (OOP) is a programming language model organized around **objects** rather than **functions** and the **logic** required for them.
- Historically, a program was viewed as a logical procedure that takes input data, processes it, and produces output data.
- The programming challenge was seen as **how to write the logic**, not **how to define the objects** in the system.
- Object-oriented programming takes the view that **what matters most are the objects** we want to manipulate, not the logic required to manipulate them.

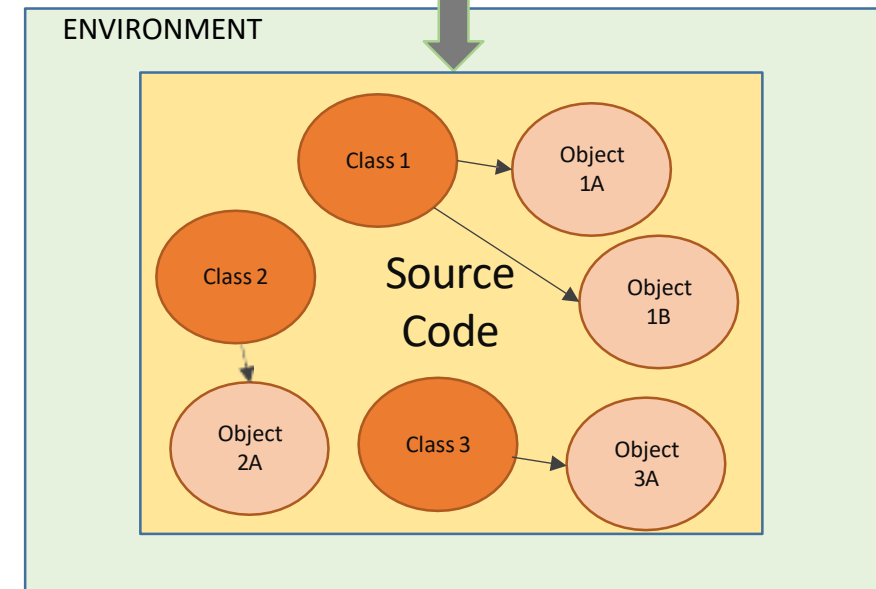
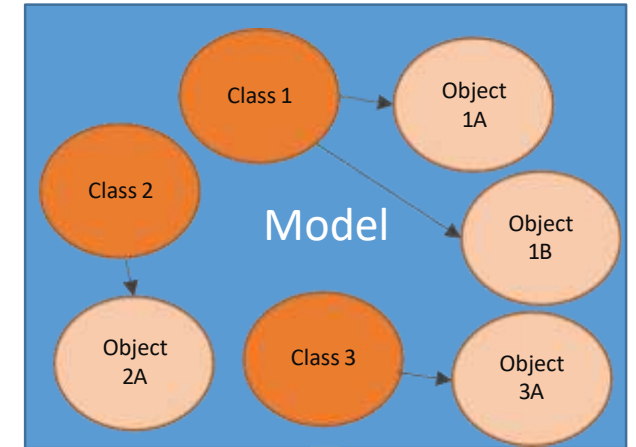


A screenshot of a Windows-style dialog box titled "Add New Employee" with a red close button in the top right corner. The dialog contains three text input fields: "Name:" with the text "Joe Bloggs" entered, "Date of Birth:", and "Phone Number:". Below the fields are two buttons: "OK" and "Cancel".

- Examples of objects range from human beings (described by name, address, etc.) to buildings and floors (whose properties can be described and managed) down to the little widgets on a computer desktop (such as buttons and scroll bars).

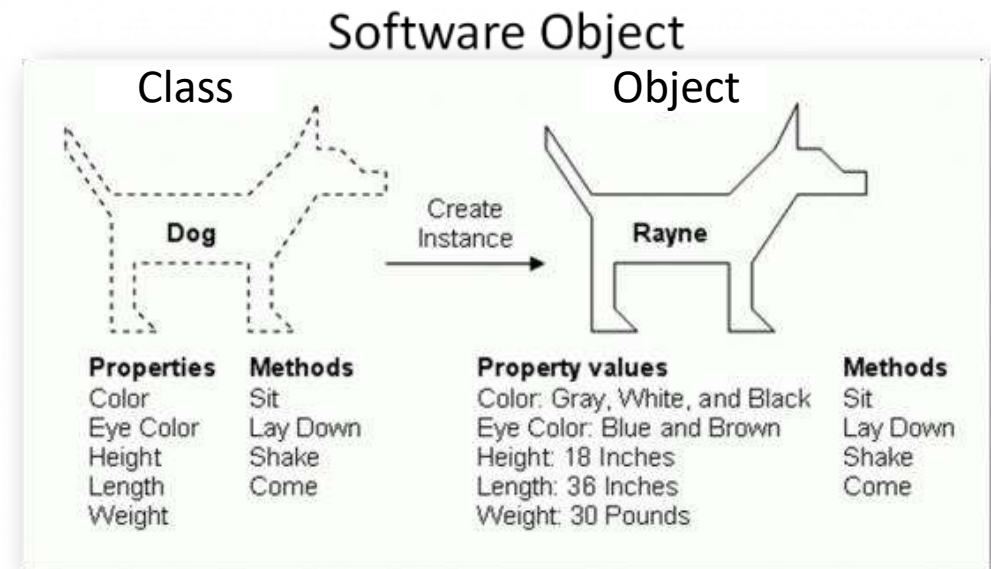
Object Oriented Construction

- Object-oriented construction involves turning the analysis model into source code, which is executed in the target environment
- The goal is that the classes and objects identified during the analysis should also be found within the code.
- This is known as **traceability**



Classes and Objects

- **Class**
 - Defines how to make an object of that type
 - Blueprint for an object
 - Has properties and methods
- **Object**
 - Instance of a class
 - Properties take values
 - These show the state of the object



Instantiation

In our chess example we identified that the class 'chess piece' has sub-classes:

- King
- Queen
- Bishop
- Knight
- Rook
- Pawn



We can create an instance of the sub-class King and call it *BlackKing*, and another instance of the sub-class and call it *WhiteKing*. We can set the 'colour' property of the black king to black and the colour property of the white king to 'white'

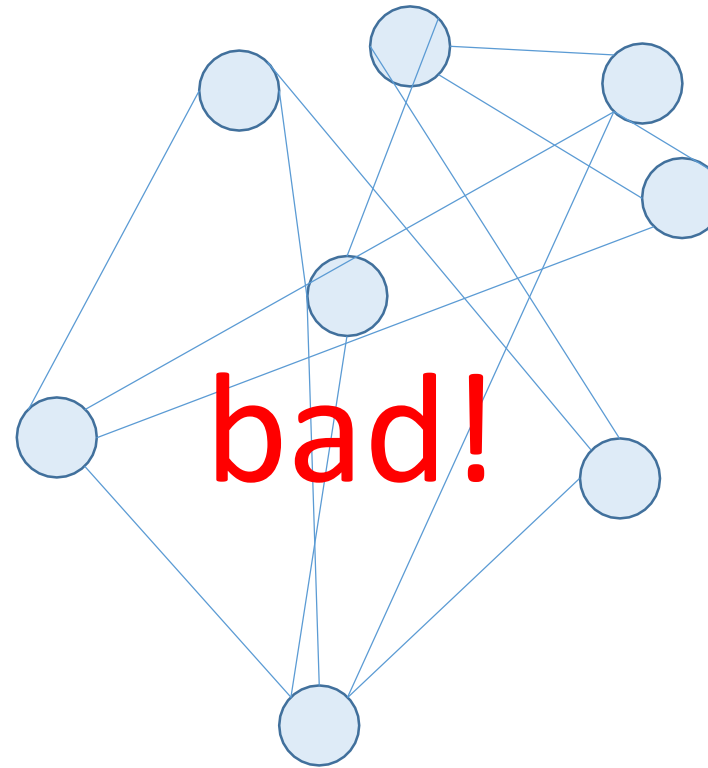
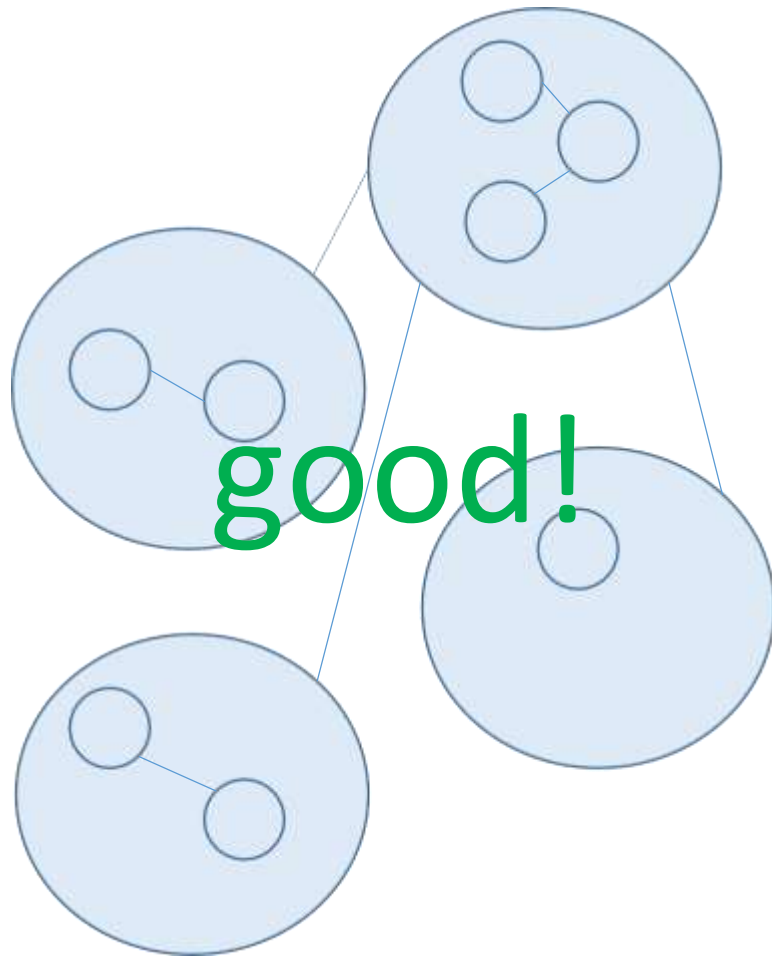
Steps in Object Oriented Modelling

1. Identify the objects – look for nouns in the system
2. Organise the objects into a class structure
3. Describe how the objects interact – think of possible use cases
4. Define the operations on the objects - methods
5. Define the objects internally - attributes



OOP and Coupling and Cohesion

Coupling and Cohesion



An established general principle of programming is that **low coupling** and **high cohesion** are good.

Coupling - To what degree does a program unit rely on other program units?

- Ideal is low coupling between units of code

○ **Low:**

- Robust
- Reliable
- Reusable
- Testable
- Understandable

○ *Evidence:*

- Dependencies on or references to other classes are few
- Parameters are few and of relatively simple types
- Few or no global variables

○ **High:**

- A change in one unit forces changes in other units
- Difficult to test
- Difficult to re-use
- Difficult to understand

○ *Evidence:*

- Dependencies on other classes are many
- Parameters are many and/or of complex types
- Many global variables

How OOP Reduces Coupling

- OOP, through the principle of encapsulation helps to create code that is loosely coupled.
- This means that an object does not directly change or modify the state or behaviour of another object.
- Objects that rely on other objects and can modify the states of other objects are said to be *tightly coupled*.
- Tight coupling creates situations where modifying the code of one object also requires changing the code of other objects (also known as a ripple effect).
- Tightly coupled code is also harder to reuse because it is difficult to separate it into the essential components

Types of Coupling

Worse

- Pathological (aka content coupling)
 - Units rely on each other's internal workings.
- Common (aka global coupling)
 - Units share common global data. E.g., units share a global variable
- External
 - Units share an externally-imposed dependency, such as a communications protocol or API
- Control
 - One unit is controlled by another
- Data-structured (aka "stamp" coupling)
 - Units share a composite data structure but each unit only uses part of it
- Data
 - Units share data through parameters
- Message
 - Units share data through message passing
- None
 - Units share nothing

Better



However ...

Units (e.g., classes, methods, modules, etc.) that share *nothing* with *anything* are the least coupled...

...but are obviously of limited value.



Cohesion - The principle of being or doing one thing well?

- **High:**

- Robust
- Reliable
- Reusable
- Understandable

- *Evidence:*

- Each class serves one purpose
- Each method performs one activity

- **Low:**

- Difficult to maintain
- Difficult to test
- Difficult to re-use
- Difficult to understand

- *Evidence:*

- Class functionality is diverse
- Methods perform many activities

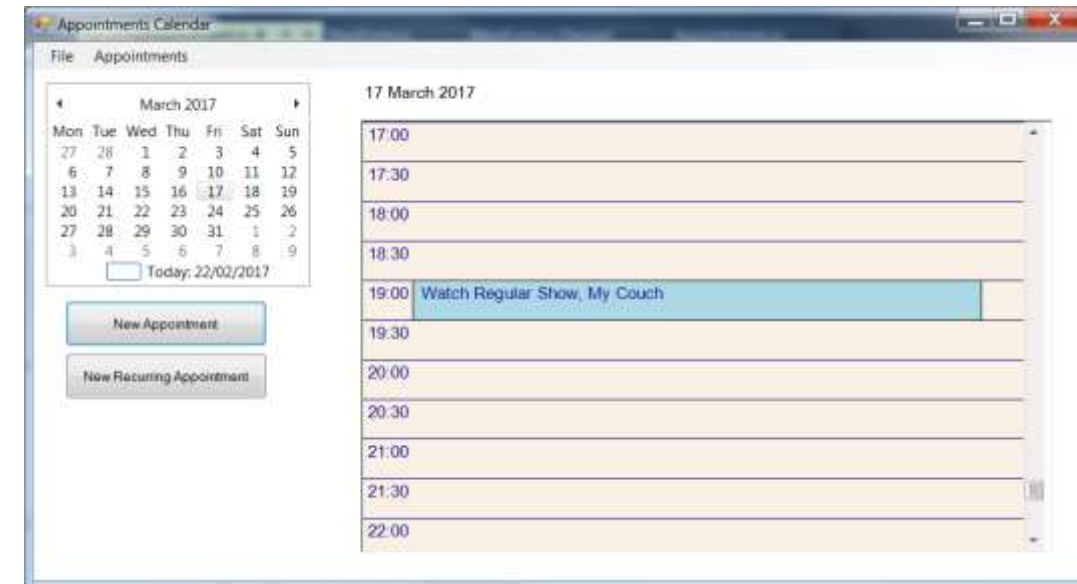
- Ideal is high cohesion within units of code

How OOP Increases Cohesion

- OOP design encourages high cohesion because it uses classes, objects, inheritance, and methods i.e. we have put thought into the relationships and interactions between objects, and their purpose and behaviours
- We are thus less likely to have generic methods that try to accomplish more than one thing

How OOP Increases Cohesion

- Ideally in OOP, each object should only have one responsibility, e.g. an *Appointments* object in a calendar application has responsibility to populate the calendar form with a list of appointments
- Every behaviour of that object should only be concerned with one task, e.g. a load method to read in the list of appointments
- If a behaviour is concerned with more than one task then it becomes more difficult to manage changes to the code



Types of Cohesion

Worse

- **Coincidental**
 - Arbitrary, e.g., a “Utility” class
- **Logical**
 - Related by conceptual category, e.g., a “UserInput” class
- **Temporal**
 - Related by when they are processed, e.g., an “ExceptionHandler” class only called on exception
- **Procedural**
 - Related by always following a set sequence, e.g., a “FileLoad” method that checks to see if a file exists and then loads a file
- **Communicational**
 - Related by working on same data, e.g., a “Player” class in a game
- **Sequential**
 - Related by the output from one unit being the input to the next. E.g., a “FileReader” method that reads data from a file and processes it.
- **Functional**
 - Related by the fact that all parts directly contribute to the same purpose. E.g., a “StringTokenizer” class.

Better

Low Cohesion



Becoming less
related by
functionality

High Cohesion



Very
Good

OOP Languages

- Simula was the first object-oriented programming language.
- Java, Python, C++, C#, Visual Basic.NET and Ruby are the most popular OOP languages today.
- Curl, Smalltalk, Delphi and Eiffel are also examples of object-oriented programming languages.

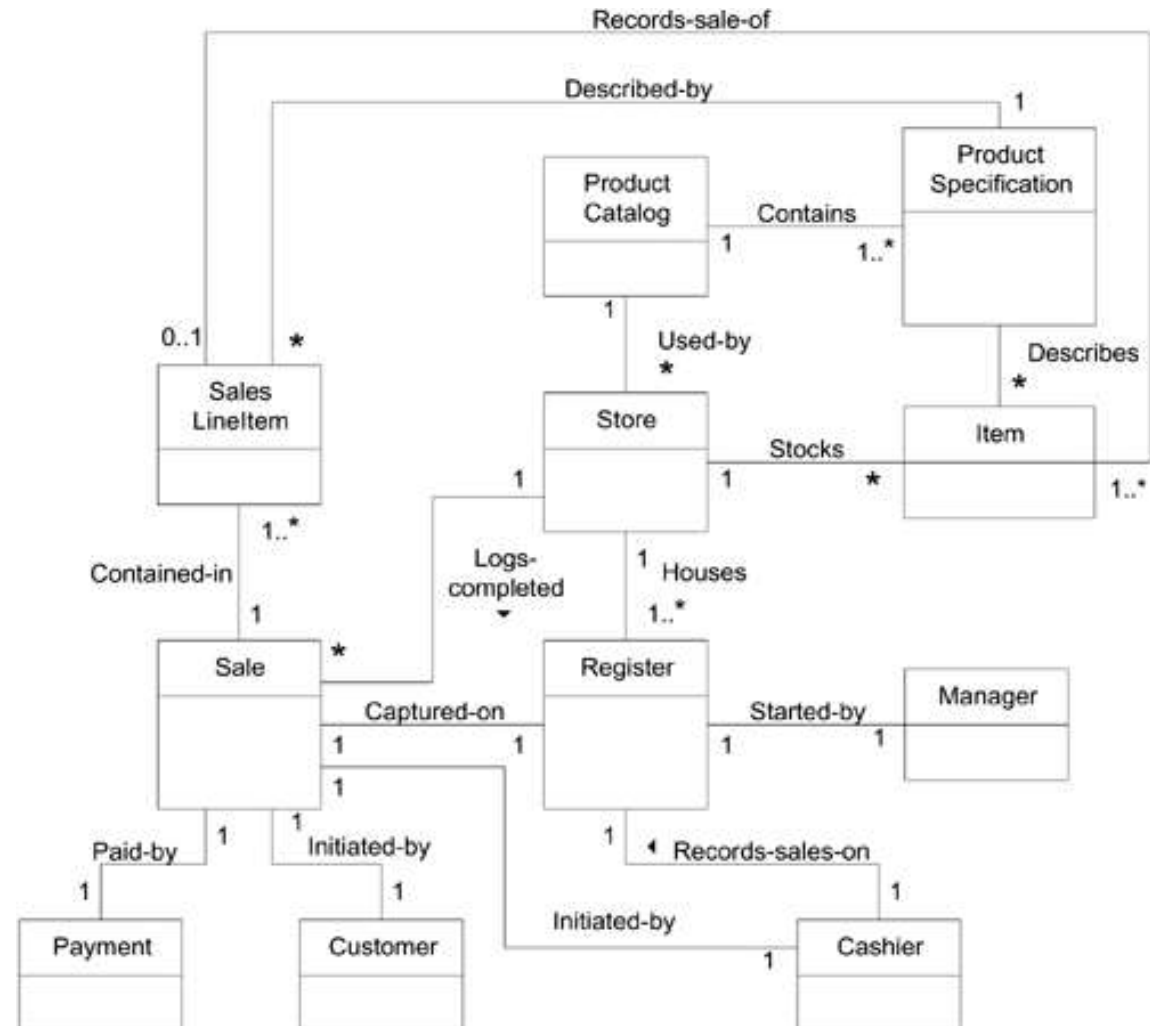


Domain Model

- The domain model is a representation of real-situation conceptual classes
- The term does not mean a set of diagram describing software classes
- Bridges the representational gap between Use-Case Model and Software Design Model (such as Class and Interaction Diagrams)

An Example: POS System Domain Model

- POS: Point of Sale System
 - A POS system is a computerized application used in part to record sales and handle payments; it is typically used in a retail store.
 - It includes hardware components such as computer and bar code scanner, and software to run the system.
 - It interfaces to various service applications, such as a third-party tax calculator and inventory control.

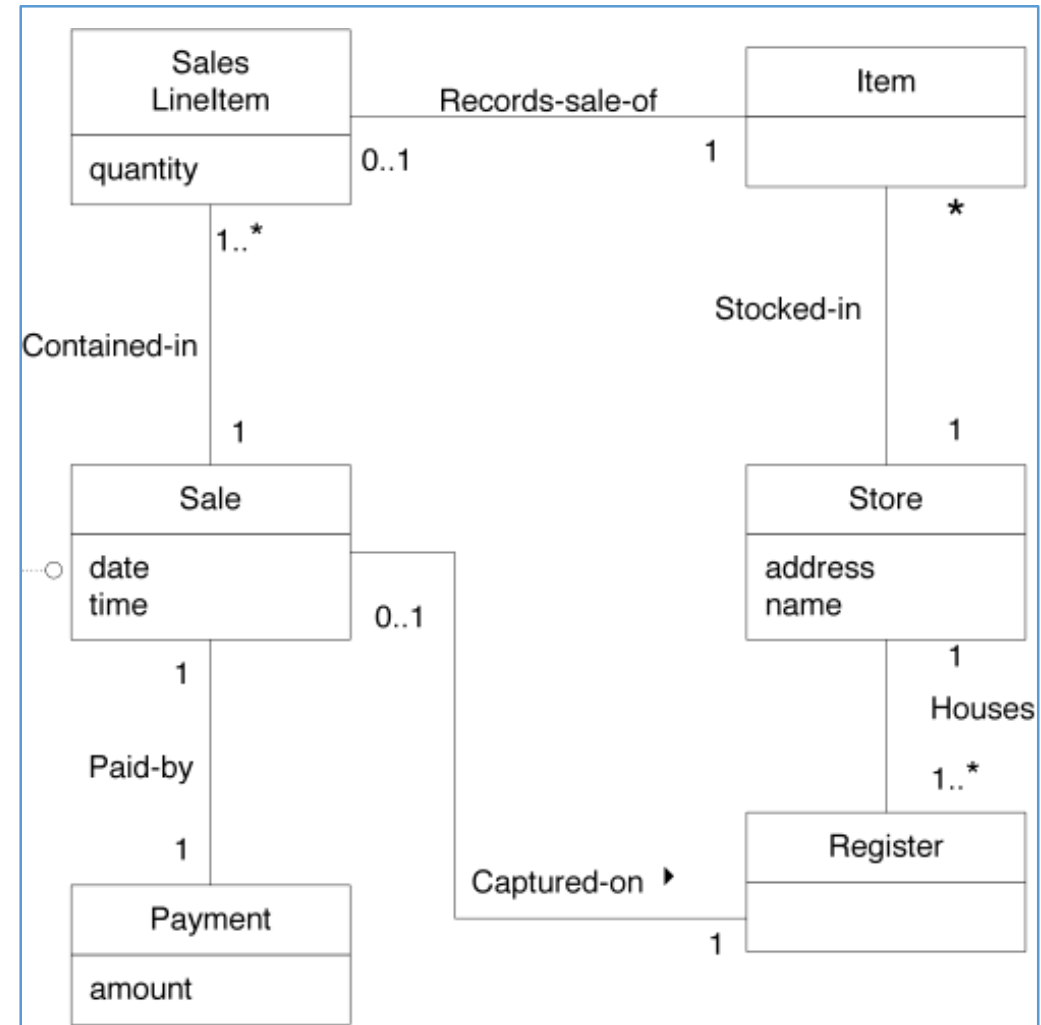


Notation – UML Class Diagram

- UML class diagram is used to model a domain with a twist
- Domain Class
 - Domain class consist of class name and attributes
 - Domain class does not have methods
- Relation
 - A line with a label represents the relation between two classes
 - A relation has a cardinality constraint

Domain Class Model

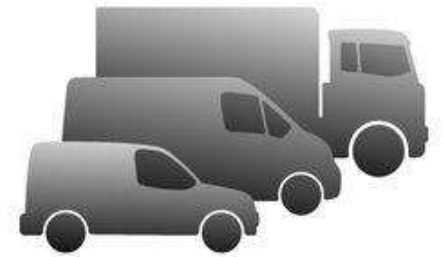
- Find conceptual classes
- Draw them as classes in a UML class diagram
- Add associations and attributes



Group Exercise

- Work in groups to create an object-oriented model of one of the following systems:
 - A library system
 - A university application and enrolment system
 - A vehicle booking system
 - The system you are working on for your Assessment project
 - Any other system that you find interesting

(25 minutes)



Summary

- By the end of this lecture, you should have begun to develop some ideas and opinions about the following:
 - The differences between functional / data modelling and analysis and object-oriented modelling and analysis
 - The advantages of object-oriented modelling and analysis
 - The OO principles of encapsulation, information hiding, inheritance and polymorphism
 - How OO programming assists with creating code that has the properties of loose coupling and high cohesion
 - The domain model as a representation of real-situation conceptual classes



Thank You!

